

Rapport Data-Challenge

(Après avoir téléchargé + combiné les données)

Table Interactive :

- Chargement des Library nécessaires (Shiny, DT, dplyr)
- Avec Shiny j'ai défini l'interface utilisateur (page web fluide, titre...)
- Avec dplyr j'ai sélectionné les variables que je voulais visualiser dans ma table à partir des données combinés (j'ai pris 10 variables)
- Rendre les données sélectionnées interactive avec DT
- Lancer l'application shiny.
- Lors de l'affichage j'avais le titre qui s'affichait en haut tout à gauche pour changer son emplacement j'ai inclus la classe CSS dans l'interface shiny.

C:\Users\PC PEO MAX\Music\Challenge - Shiny
http://127.0.0.1:3940 Open in Browser Publish

Données COVID-19										
Show	10	entries	Search:							
	source_file	Country_Region	Province_State	Last_Update	Confirmed	Deaths	Recovered	Active	Incident_Rate	Case_Fatality_Ratio
1	01-01-2021	Afghanistan		2021-01-02 05:22:33	52513	2201	41727	8585	134.8965783052507	4.191343095995277
2	01-01-2021	Albania		2021-01-02 05:22:33	58316	1181	33634	23501	2026.409062478282	2.025173194320598
3	01-01-2021	Algeria		2021-01-02 05:22:33	99897	2762	67395	29740	227.8098607597543	2.764847793227024
4	01-01-2021	Andorra		2021-01-02 05:22:33	8117	84	7463	570	10505.40348152462	1.03486509794259
5	01-01-2021	Angola		2021-01-02 05:22:33	17568	405	11146	6017	53.45298103210258	2.305327868852459
6	01-01-2021	Antigua and Barbuda		2021-01-02 05:22:33	159	5	148	6	162.3641859325218	3.144654088050315
7	01-01-2021	Argentina		2021-01-02 05:22:33	1629594	43319	1426676	159599	3605.633331627422	2.658269483073698
8	01-01-2021	Armenia		2021-01-02 05:22:33	159738	2828	143355	13555	5390.664388975018	1.77039902840902
9	01-01-2021	Australia	Australian Capital Territory	2021-01-02 05:22:33	118	3	114	1	27.56365335202056	2.542372881355932
10	01-01-2021	Australia	New South Wales	2021-01-02 05:22:33	4947	54	0	4893	60.93865484109386	1.091570648878108
Showing 1 to 10 of 76,274 entries				Previous	1	2	3	4	5	... 7,628 Next

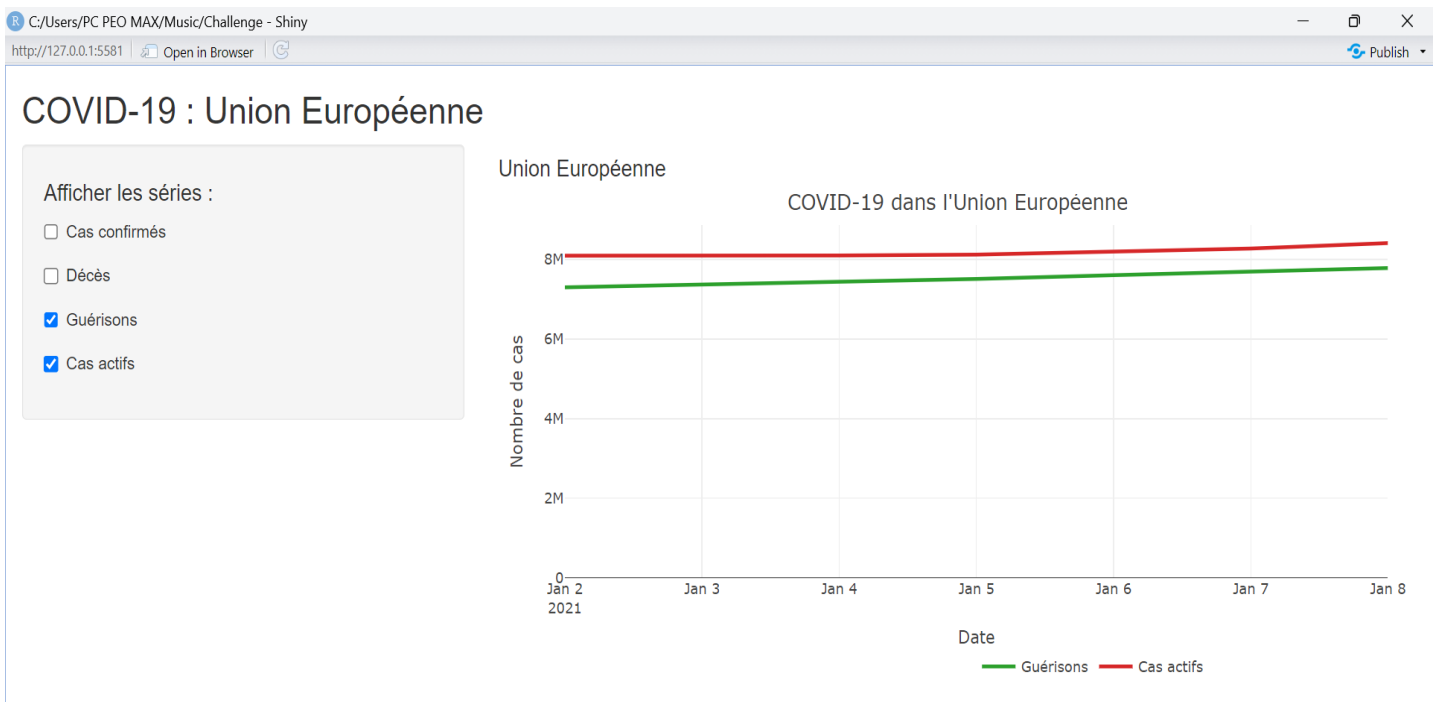
Graphique : Évolution dans le temps du covid dans l'Union Européenne

- Library nécessaire : Shiny, dplyr, plotly
- Liste des pays de l'union européenne + appliquer le filtre
- Interface utilisateur avec shiny
- Choix des variable à visualiser + la somme de chaque variable + agrégation groupe by (Last_Update)
- Assemblage des graphiques avec plotly
- Lancer l'application

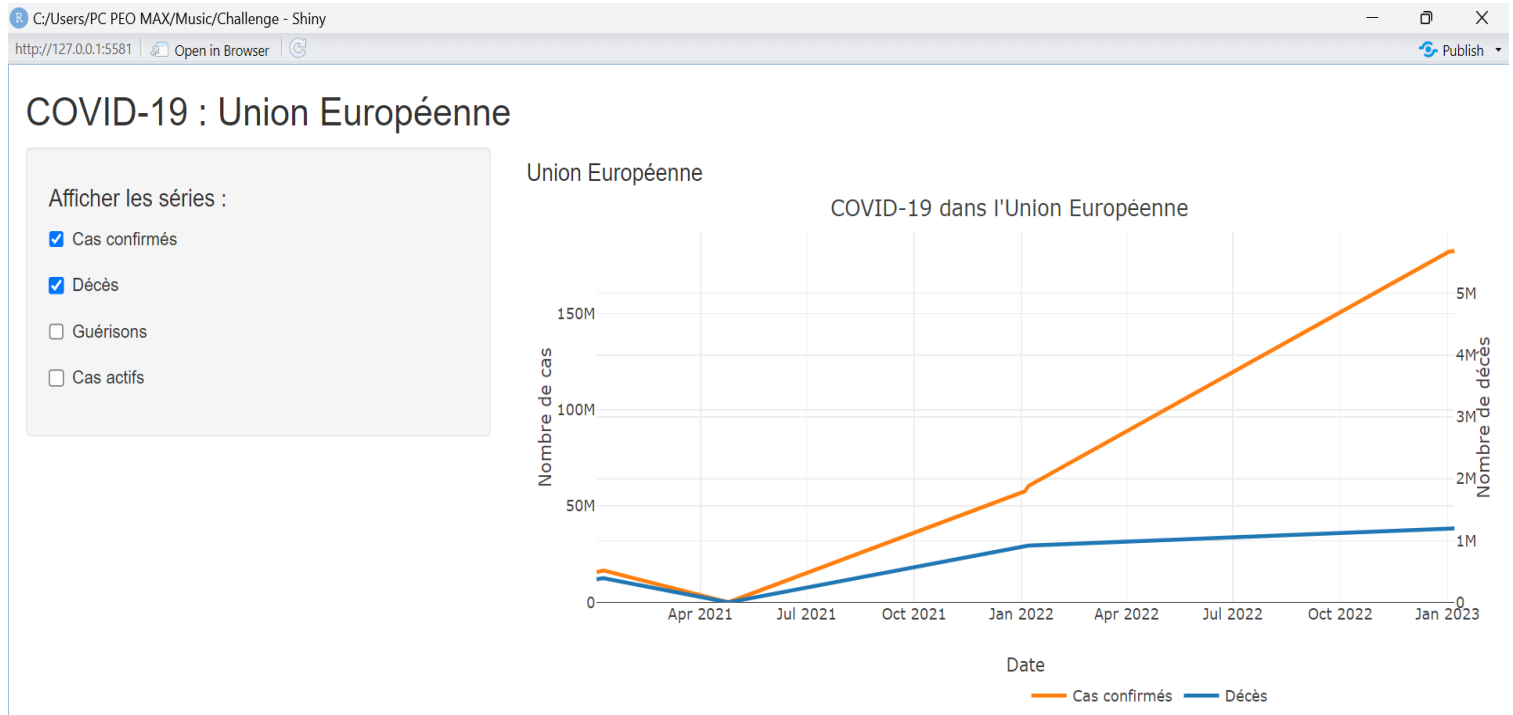
###Obstacles###

Les variables choisies : Deaths (Décès), Recovered (Guérison), Active (Cas actif), Confirmed (Cas confirmé).

- 1) Pour les pays de l'UE le nombre de Guérisons et de Cas actifs contient que des NA à partir du 8 janvier 2021, j'allais remplacer les NA par 0 (Recovered = ifelse(is.na(Recovered), 0, Recovered) / Active = ifelse(is.na(Active), 0, Active))
Mais j'ai préféré les garder comme tel car c'est comme si je disais que le nombre de guérison est de 0 alors qu'en fait il est juste inconnu.

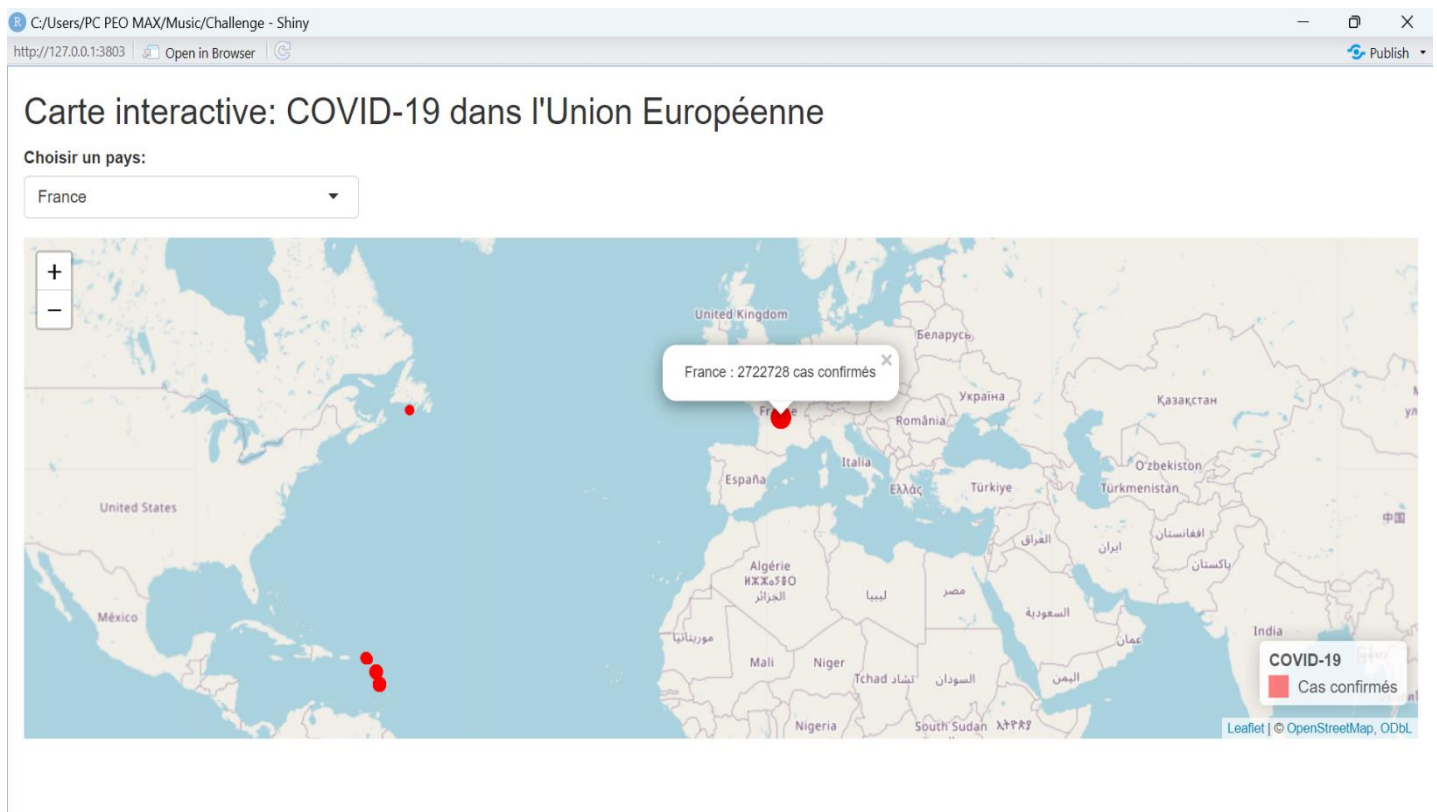


- 2) Le nombre de Décès et le nombre de cas confirmés n'ont pas les mêmes ordres de grandeur (échelle différente !!) donc création d'un nouvel axe Y2 pour le nombre de décès pour une meilleure visualisation



Carte Interactive : Visualiser les cas confirmés de COVID-19 dans les pays de l'Union Européenne (UE) sur une carte.

- Charger les Library : Shiny, leaflet, dplyr
- Sélection des pays de l'union européenne et les mettre dans un vecteur eu_countries puis filtrer les données pour ne prendre que les pays de l'union européenne à partir des données combinées.
- Interface utilisateur avec shiny (page web, titre, recherche par pays/ par défaut France)
- Générer la carte avec Leaflet (légende : cas confirmés couleur rouge, choix de la taille des cercles...ect)
- Lancer l'application Shiny



####Code####

####Combinées les données####

```
library(dplyr)
```

```
# Fonction pour charger les fichiers CSV téléchargés
```

```
load_covid_reports <- function(base_path = "covid_data") {
```

```
  # Lister tous les fichiers CSV dans le dossier
```

```
  csv_files <- list.files(base_path, pattern = "\\*.csv$", full.names = TRUE)
```

```
  message(sprintf("Found %d CSV files in the folder", length(csv_files)))
```

```
  # Initialiser une liste pour stocker les data.frames
```

```
  covid_data_list <- list()
```

```
  # Lire chaque fichier CSV et les stocker dans la liste
```

```
  for (file_path in csv_files) {
```

```
    # Lire le fichier CSV dans un data.frame
```

```
    covid_data <- read.csv(file_path)
```

```
    # Ajouter le data.frame à la liste avec un nom basé sur le nom du fichier
```

```
    table_name <- gsub(".csv$", "", basename(file_path))
```

```
    covid_data_list[[table_name]] <- covid_data
```

```
  message(sprintf("Loaded data from %s", basename(file_path)))
```

```

}

# Combiner tous les data.frames en un seul data.frame
combined_covid_data <- bind_rows(covid_data_list, .id = "source_file")

message("All data loaded and combined into a single data frame.")

return(combined_covid_data)
}

base_path <- "covid_data"
combined_data <- load_covid_reports(base_path)

# Voir un aperçu des données combinées
head(combined_data)

#####Code#####
#####Table#####

library(shiny)
library(DT)
library(dplyr)

ui <- fluidPage(
  titlePanel("Données COVID-19"),
  sidebarLayout(
    sidebarPanel(
    ),
    mainPanel(
      DTOutput("table_data")
    )
  )
)

server <- function(input, output) {

  selected_data <- combined_data %>%
    select(
      source_file,
      Country_Region,
      Province_State,
      Last_Update,
      Confirmed,
      Deaths,
      Recovered,
      Active,
      Incident_Rate,
      Case_Fatality_Ratio
    )

  output$table_data <- renderDT({

```

```

    datatable(
      selected_data,
      options = list(pageLength = 10, searchHighlight = TRUE),
      rownames = TRUE
    )
  })
}

```

```

# Lancer l'application Shiny
shinyApp(ui, server)

```

```

#####Code#####
#####Graphique#####

```

```

library(shiny)
library(plotly)
library(dplyr)

```

```

eu_countries <- c("France", "Germany", "Italy", "Spain", "Poland", "Romania",
  "Netherlands", "Belgium", "Czechia", "Greece", "Portugal",
  "Sweden", "Hungary", "Austria", "Bulgaria", "Denmark",
  "Finland", "Slovakia", "Ireland", "Croatia", "Lithuania",
  "Slovenia", "Latvia", "Estonia", "Cyprus", "Luxembourg", "Malta")

```

```

ui <- fluidPage(
  titlePanel("COVID-19 : Union Européenne"),
  sidebarLayout(
    sidebarPanel(
      h4("Afficher les séries :"),
      checkboxInput("show_confirmed", "Cas confirmés", value = TRUE),
      checkboxInput("show_deaths", "Décès", value = TRUE),
      checkboxInput("show_recovered", "Guérisons", value = TRUE),
      checkboxInput("show_active", "Cas actifs", value = TRUE)
    ),
    mainPanel(
      h4("Union Européenne"),
      plotlyOutput("plot_eu")
    )
  )
)

```

```

server <- function(input, output) {

  combined_data$Last_Update <- as.Date(combined_data$Last_Update, format =
"%Y-%m-%d")

  eu_data <- combined_data %>%
    filter(Country_Region %in% eu_countries) %>%
    filter(Last_Update >= as.Date("2020-08-04")) %>%

```

```

group_by(Last_Update) %>%
summarise(
  Total_Confirmed = sum(Confirmed),
  Total_Deaths = sum(Deaths),
  Total_Recovered = sum(Recovered),
  Total_Active = sum(Active)
) %>%
ungroup()

output$plot_eu <- renderPlotly({
  p <- plot_ly(eu_data, x = ~Last_Update)

  if (input$show_confirmed) {
    p <- p %>% add_lines(y = ~Total_Confirmed, name = "Cas confirmés", line =
list(color = "#FF7F0E", width = 3))
  }
  if (input$show_deaths) {
    p <- p %>% add_lines(y = ~Total_Deaths, name = "Décès", yaxis = "y2",
      line = list(color = "#1F77B4", width = 3))
  }
  if (input$show_recovered) {
    p <- p %>% add_lines(y = ~Total_Recovered, name = "Guérisons", line =
list(color = "#2CA02C", width = 3))
  }
  if (input$show_active) {
    p <- p %>% add_lines(y = ~Total_Active, name = "Cas actifs", line = list(color =
"#D62728", width = 3))
  }

  p <- p %>% layout(
    title = "COVID-19 dans l'Union Européenne",
    xaxis = list(title = "Date"),
    yaxis = list(
      title = "Nombre de cas",
      rangemode = "tozero"
    ),
    yaxis2 = list(
      title = "Nombre de décès",
      overlaying = "y",
      side = "right",
      rangemode = "tozero",
      range = c(0, max(eu_data$Total_Deaths) * 5) # Ajuste la hauteur
    ),
    legend = list(orientation = "h", x = 0.5, y = -0.2),
    plot_bgcolor = "#FFFFFF",
    paper_bgcolor = "#FFFFFF"
  )
})

```

```
    p
  })
}
```

```
# Lancer l'application
shinyApp(ui, server)
```

```
#####Code#####
```

```
#####Carte#####
```

```
library(shiny)
library(leaflet)
library(dplyr)
```

```
eu_countries <- c("France", "Germany", "Italy", "Spain", "Poland", "Romania",
                  "Netherlands", "Belgium", "Czechia", "Greece", "Portugal",
                  "Sweden", "Hungary", "Austria", "Bulgaria", "Denmark",
                  "Finland", "Slovakia", "Ireland", "Croatia", "Lithuania",
                  "Slovenia", "Latvia", "Estonia", "Cyprus", "Luxembourg", "Malta")
```

```
eu_data <- combined_data %>%
  filter(Country_Region %in% eu_countries) %>%
  select(Country_Region, Lat, Long_, Confirmed)
```

```
ui <- fluidPage(
  titlePanel("Carte interactive COVID-19 de l'Union Européenne"),
```

```
  selectInput("country_select", "Choisir un pays:", choices = eu_countries, selected =
"France"),
```

```
  leafletOutput("covid_map")
)
```

```
server <- function(input, output, session) {
```

```
  filtered_data <- reactive({
    combined_data %>%
      filter(Country_Region == input$country_select)
  })
```

```
  output$covid_map <- renderLeaflet({
    data <- filtered_data()
```



```

leaflet(data) %>%
  addTiles() %>%
  addCircleMarkers(
    lng = ~Long_, lat = ~Lat,
    radius = ~log(Confirmed + 1)/2,
    color = "red",
    stroke = FALSE,
    fillOpacity = 0.6,
    popup = ~paste(Country_Region, ": ", Confirmed, " cas confirmés")
  ) %>%
  addLegend(
    position = "bottomright",
    colors = c("red"),
    labels = c("Cas confirmés"),
    title = "COVID-19"
  ) %>%
  setView(lng = mean(data$Long_), lat = mean(data$Lat), zoom = 5) # Centrer sur
le pays sélectionné
  })
}

# Lancer l'application Shiny
shinyApp(ui = ui, server = server)

```